

# Lorentz System Dynamics

The Lorentz system is a simplified mathematical model for atmospheric convection that exhibits chaotic behavior. Originally derived by Edward Lorentz in 1963 as a simplified model of atmospheric convection, it has become one of the most studied examples of chaos theory and nonlinear dynamics.

From a physics perspective, the Lorentz equations emerge from a dramatic simplification of the Navier-Stokes equations governing fluid flow, specifically applied to Rayleigh-Bénard convection - the phenomenon where a fluid layer heated from below develops convective rolls. In this physical context, the three variables have clear meanings:  $x$  represents the rate of convective overturning,  $y$  corresponds to the horizontal temperature difference between ascending and descending fluid parcels, and  $z$  relates to the deviation of the vertical temperature profile from linearity.

The system parameters also have physical significance:  $\sigma$  is the Prandtl number (ratio of momentum to thermal diffusivity),  $\rho$  is proportional to the Rayleigh number (measuring the strength of thermal driving), and  $\beta$  relates to the geometry of the convection cells. This makes the Lorentz system not just a mathematical curiosity, but a window into understanding how deterministic physical systems can exhibit unpredictable behavior - a discovery that revolutionized meteorology and our understanding of complex systems in physics.

The emergence of chaos in such a simple system has profound implications for physics, demonstrating that even classical, deterministic systems can be fundamentally unpredictable due to sensitive dependence on initial conditions - challenging the Laplacean view of a clockwork universe and establishing chaos as a fundamental feature of nonlinear physics.

```
#| edit: false
#| echo: false
#| execute: true

import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

```
# Set default plotting parameters
plt.rcParams.update({
    'font.size': 12,
    'lines.linewidth': 1,
    'lines.markersize': 5,
    'axes.labelsize': 11,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'xtick.top': True,
    'xtick.direction': 'in',
    'ytick.right': True,
    'ytick.direction': 'in',
})

def get_size(w, h):
    return (w/2.54, h/2.54)
```

## Derivation of the Lorentz Equations

The Lorentz system emerges from a dramatic simplification of the fluid dynamics equations governing thermal convection. The derivation begins with the Rayleigh-Bénard convection problem: a thin layer of fluid heated uniformly from below and cooled from above.

### From Navier-Stokes to Lorentz

The full problem is described by the **Navier-Stokes equations** for fluid motion coupled with the **heat equation** for temperature transport:

- **Momentum equation:**  $\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{v} + \mathbf{g} \alpha T$
- **Continuity equation:**  $\nabla \cdot \mathbf{v} = 0$
- **Heat equation:**  $\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \kappa \nabla^2 T$

where  $\mathbf{v}$  is the velocity field,  $p$  is pressure,  $T$  is temperature,  $\nu$  is kinematic viscosity,  $\kappa$  is thermal diffusivity,  $\alpha$  is thermal expansion coefficient, and  $\mathbf{g}$  is gravitational acceleration.

### The Key Simplification

Lorentz made a crucial simplification by expanding the velocity and temperature fields in **Fourier modes** and keeping only the most essential modes:

- The velocity field is approximated by a **single convection roll pattern**
- The temperature field is described by just **two modes**: one representing the basic temperature gradient and another representing the convective disturbance

This reduces the infinite-dimensional partial differential equation system to just **three ordinary differential equations** for the amplitudes of these modes.

## The Resulting System

The Lorentz system consists of three coupled nonlinear ordinary differential equations:

$$\frac{dx}{dt} = \sigma(y - x) \tag{1}$$

$$\frac{dy}{dt} = x(\rho - z) - y \tag{2}$$

$$\frac{dz}{dt} = xy - \beta z \tag{3}$$

where  $\sigma$ ,  $\rho$ , and  $\beta$  are system parameters derived from the physical properties of the fluid.

## Physical Interpretation of Each Equation

**First equation** ( $\frac{dx}{dt} = \sigma(y - x)$ ):

- Describes how the **convection rate**  $x$  responds to the horizontal temperature difference  $y$
- The term  $\sigma(y - x)$  represents the tendency for convection to develop when there's a temperature gradient
- $\sigma$  is the Prandtl number ( $\nu/\kappa$ ), measuring the relative importance of momentum vs. thermal diffusion

**Second equation** ( $\frac{dy}{dt} = x(\rho - z) - y$ ):

- Governs the **horizontal temperature variation**  $y$
- The term  $x(\rho - z)$  shows how convection ( $x$ ) driven by the temperature gradient ( $\rho - z$ ) creates horizontal temperature differences
- The  $-y$  term represents the natural decay of temperature differences due to thermal conduction
- $\rho$  is proportional to the Rayleigh number, measuring the strength of thermal driving

**Third equation** ( $\frac{dz}{dt} = xy - \beta z$ ):

- Controls the **vertical temperature profile deviation**  $z$  from the linear conductive state
- The term  $xy$  represents how convection creates vertical temperature variations
- The  $-\beta z$  term describes the relaxation back to the conductive profile
- $\beta$  relates to the aspect ratio of the convection cell geometry

This elegant reduction captures the essential physics of thermal convection while remaining simple enough to analyze mathematically, yet complex enough to exhibit the rich chaotic dynamics that Lorentz discovered.

## System Parameters

The classic parameters that lead to chaotic behavior are:

- $\sigma = 10$  (Prandtl number)
- $\rho = 28$  (Rayleigh number)
- $\beta = 8/3$  (geometric factor)

```
#| autorun: false

# Define the Lorentz system parameters
sigma = 10.0
rho = 28.0
beta = 8.0/3.0

def lorentz_system(t, state):
    """
    Lorentz system of equations
    """
    x, y, z = state

    dxdt = sigma * (y - x)
    dydt = x * (rho - z) - y
    dzdt = x * y - beta * z

    return [dxdt, dydt, dzdt]
```

## Solving the System

We solve the system using numerical integration starting from different initial conditions:

```

#| autorun: false

# Time span
t_span = (0, 40)
t_eval = np.linspace(0, 40, 4000)

# Initial conditions
initial_conditions = [
    [1.0, 1.0, 1.0],
    [1.0, 1.0, 1.01], # Slightly perturbed
    [0.0, 1.0, 1.05],
    [-8.0, 8.0, 27.0]
]

# Solve the system for each initial condition
solutions = []
for ic in initial_conditions:
    sol = solve_ivp(lorentz_system, t_span, ic, t_eval=t_eval,
                    method='RK45', rtol=1e-8)
    solutions.append(sol)

```

## Visualization

### 3D Phase Portrait

The famous “butterfly attractor” emerges when we plot the trajectory in 3D phase space:

```

#| autorun: false

fig = plt.figure(figsize=get_size(12, 10))
ax = fig.add_subplot(111, projection='3d')

# Plot trajectories
colors = ['blue', 'red', 'green', 'orange']
for i, (sol, color) in enumerate(zip(solutions, colors)):
    ax.plot(sol.y[0], sol.y[1], sol.y[2], color=color, alpha=0.7,
            label=f'IC {i+1}')

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')

```

```
#ax.set_title('Lorentz Attractor - 3D Phase Portrait')
#ax.legend()

plt.tight_layout()
plt.show()
```

## Time Series

Let's examine the time evolution of each variable:

```
#| autorun: false

fig, axes = plt.subplots(3, 1, figsize=get_size(12, 10))

sol = solutions[0] # Use first solution
t = sol.t

# Plot x, y, z vs time
axes[0].plot(t, sol.y[0], 'b-', alpha=0.8)
axes[0].set_ylabel('x(t)')
axes[0].grid(True)

axes[1].plot(t, sol.y[1], 'r-', alpha=0.8)
axes[1].set_ylabel('y(t)')
axes[1].grid(True)

axes[2].plot(t, sol.y[2], 'g-', alpha=0.8)
axes[2].set_ylabel('z(t)')
axes[2].set_xlabel('Time')
axes[2].grid(True)

plt.tight_layout()
plt.show()
```

## Sensitive Dependence on Initial Conditions

One hallmark of chaos is sensitive dependence on initial conditions. Let's compare two trajectories with nearly identical starting points:

```

#| autorun: false

# Compare first two solutions (very similar initial conditions)
sol1, sol2 = solutions[0], solutions[1]

fig, axes = plt.subplots(2, 1, figsize=get_size(12, 8))

# Plot x coordinate for both solutions
axes[0].plot(sol1.t, sol1.y[0], 'b-', label='IC: (1.0, 1.0, 1.0)', alpha=0.8)
axes[0].plot(sol2.t, sol2.y[0], 'r-', label='IC: (1.0, 1.0, 1.01)', alpha=0.8)
axes[0].set_ylabel('x(t)')
axes[0].legend()
axes[0].grid(True)
#axes[0].set_title('Sensitive Dependence on Initial Conditions')

# Plot difference between solutions
diff = np.abs(sol1.y[0] - sol2.y[0])
axes[1].semilogy(sol1.t, diff, 'k-', alpha=0.8)
axes[1].set_ylabel('|x(t) - x(t)|')
axes[1].set_xlabel('Time')
axes[1].grid(True)
#axes[1].set_title('Exponential Divergence')

plt.tight_layout()
plt.show()

```

## Poincaré Section

A useful way to analyze chaotic systems is through Poincaré sections, which show where trajectories intersect a chosen plane:

```

#| autorun: false

# Create Poincaré section at  $z = \rho - 1$ 
z_section = rho - 1

def find_poincare_section(sol, z_val):
    """Find points where trajectory crosses  $z = z\_val$  with  $dz/dt > 0$ """
    x, y, z = sol.y
    t = sol.t

```

```

# Find crossings
crossings_x = []
crossings_y = []

for i in range(len(t)-1):
    if z[i] < z_val and z[i+1] > z_val:
        # Linear interpolation to find exact crossing point
        alpha = (z_val - z[i]) / (z[i+1] - z[i])
        x_cross = x[i] + alpha * (x[i+1] - x[i])
        y_cross = y[i] + alpha * (y[i+1] - y[i])
        crossings_x.append(x_cross)
        crossings_y.append(y_cross)

return np.array(crossings_x), np.array(crossings_y)

# Calculate Poincaré section
sol = solutions[0]
x_poincare, y_poincare = find_poincare_section(sol, z_section)

plt.figure(figsize=get_size(10, 8))
plt.scatter(x_poincare, y_poincare, s=1, alpha=0.6)
plt.xlabel('x')
plt.ylabel('y')
plt.title(f'Poincaré Section at z = {z_section:.1f}')
plt.grid(True)
plt.show()

```

## **i** Mathematical Properties

### **Fixed Points**

The Lorentz system has three fixed points:

1. **Origin:**  $(0, 0, 0)$  - unstable for  $\rho > 1$
2. **Symmetric points:**  $(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$  - unstable for  $\rho > \sigma \frac{\sigma+\beta+3}{\sigma-\beta-1}$

### **Lyapunov Exponents**

For the classic parameters, the Lyapunov exponents are approximately:

- $\lambda_1 \approx 0.906$  (positive - indicates chaos)
- $\lambda_2 \approx 0$  (near zero)
- $\lambda_3 \approx -14.6$  (negative - indicates contraction)



The positive Lyapunov exponent confirms the chaotic nature of the system.

### Fractal Dimension

The Lorenz attractor has a fractal dimension of approximately 2.06, indicating it's neither a simple curve nor a surface but something in between.

## Parameter Exploration

Let's see how the system behavior changes with different parameter values:

```
#| autorun: false

# Test different rho values
rho_values = [10, 20, 28, 35]
fig, axes = plt.subplots(2, 2, figsize=get_size(12, 10), subplot_kw={'projection': '3d'})
axes = axes.ravel()

for i, rho_test in enumerate(rho_values):
    # Solve with different rho
    def lorentz_test(t, state):
        x, y, z = state
        dxdt = sigma * (y - x)
        dydt = x * (rho_test - z) - y
        dzdt = x * y - beta * z
        return [dxdt, dydt, dzdt]

    sol = solve_ivp(lorentz_test, (0, 30), [1.0, 1.0, 1.0],
                    t_eval=np.linspace(0, 30, 3000), method='RK45')

    axes[i].plot(sol.y[0], sol.y[1], sol.y[2], 'b-', alpha=0.7)
    axes[i].set_title(f' = {rho_test}')
    axes[i].set_xlabel('x')
    axes[i].set_ylabel('y')
    axes[i].set_zlabel('z')

plt.tight_layout()
plt.show()
```

## Applications and Significance

The Lorentz system has profound implications:

1. **Weather Prediction:** Demonstrates why long-term weather forecasting is fundamentally limited
2. **Chaos Theory:** Provides a simple example of deterministic chaos
3. **Nonlinear Dynamics:** Serves as a paradigm for studying complex systems
4. **Numerical Methods:** Tests the accuracy of numerical integration schemes

The “butterfly effect” - the idea that small changes can have large consequences - originated from Lorentz’s work on this system.

## Exercises

1. **Parameter Sensitivity:** Investigate how the system behaves for different values of  $\sigma$  and  $\beta$
2. **Initial Condition Dependence:** Quantify how quickly nearby trajectories diverge
3. **Periodic Orbits:** Find parameter values that produce periodic rather than chaotic behavior
4. **Numerical Precision:** Study how numerical errors affect long-term predictions